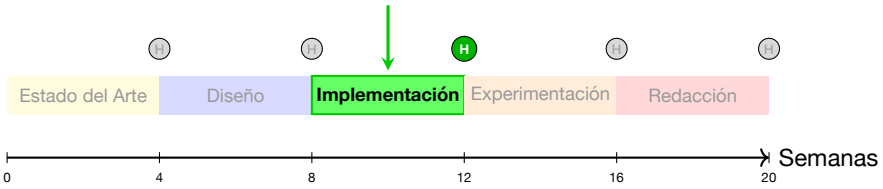
Documento de mes 3 — Implementación



Implementación: Del Prototipo al Fitness Real con Spark RDD

TFM: Selección Automática de Características en Algoritmos de Clasificación Mediante Algoritmos Genéticos y Métodos de Envoltorio con Apache Spark RDD

Este documento recoge los hitos 1 a 6 del plan de trabajo: desde la implementación del fitness real con GNB (Hito 1) hasta la primera versión completa del GA con criterio de parada dual (Hito 6).

Documentos internos referenciados

A lo largo del texto se cita un conjunto de documentos internos del proyecto. La tabla siguiente define las abreviaturas usadas; todos los ficheros se encuentran en el directorio `documentacion/` del repositorio.

Abrev.	Fichero	Descripción
[D1]	`diseño_GA.md`	Diseño del GA: -fitness §2 -operadores §3 -clasificador §5 -parámetros §6
[D2]	`arquitectura_spark_rdd_ga.md`	Arquitectura Spark RDD: -Driver/Workers §2.1 -`parallelize`→`map`→`collect` §2.3 -broadcast §2.5
[D3]	`prototipo_evaluacion_paralela.md`	Prototipo mes 2: -fitness simulado con `time.sleep` §3.2

Resumen de hitos cubiertos

Hito	Título	Notebook
1	Implementación del fitness real (GNB)	`02_gnb_fitness_evaluation.ipynb` §4–§6
2	Integración del conjunto de datos	`02_gnb_fitness_evaluation.ipynb` §2
3	Transmisión del dataset en Spark	`02_gnb_fitness_evaluation.ipynb` §3–§4
4	Evaluación paralela del fitness real	`02_gnb_fitness_evaluation.ipynb` §5–§10
5	Experimento inicial de *speed-up*	`03_speedup_experiment.ipynb` §4–§6
6	Primera versión completa del GA	`04_ga_full_version.ipynb` §5–§8

---

1. Hito 1: Implementación del Fitness Real con GNB

1.1 Objetivo

Sustituir la función *dummy* del prototipo por una evaluación real basada en un clasificador de aprendizaje automático. El clasificador elegido es **Gaussian Naive Bayes** (GNB) de scikit-learn, entrenado sobre el subconjunto de características activas del cromosoma y evaluado mediante validación cruzada estratificada de 5 pliegues (**[D1]** §5.1).

1.2 Fórmula de fitness

La función de fitness implementada sigue exactamente la especificación de **[D1]** §2:

$$F(\mathbf{c}) = \alpha \cdot \text{Accuracy}_{CV5}(\mathbf{c}) - \beta \cdot \frac{|S|}{D}$$

Con los parámetros de **[D1]** §6 (columna WDBC): $\alpha = 0.9$, $\beta = 0.1$, $k = 5$ pliegues. El término $|S|/D$ penaliza la complejidad del subconjunto, favoreciendo soluciones más parsimoniosas.

1.3 Implementación

```
def evaluar_fitness(cromosoma):
    indices_activos = [i for i, gen in enumerate(cromosoma) if gen == 1]
    if len(indices_activos) == 0:
        return 0.0                          # cromosoma nulo → fitness 0
    X_data = X_bc.value[:, indices_activos] # columnas activas via broadcast
    y_data = y_bc.value
    gnb = GaussianNB()
    scores = cross_val_score(gnb, X_data, y_data, cv=CV_FOLDS, scoring='accuracy')
    return round(ALPHA * scores.mean() - BETA * len(indices_activos) / len(cromosoma), 6)
```

Decisiones de implementación:

Decisión	Justificación
GNB sin hiperparámetros	Velocidad $\mathcal{O}(N \times D)$; sin ajuste interno que introduzca variabilidad (**[D1]** §5.1)
`StratifiedKFold` implícito	`cross_val_score` mantiene la proporción de clases; relevante para WDBC (desbalanceado: 357 vs. 212)
Cromosoma nulo → fitness 0	Penaliza soluciones inviables sin que el clasificador falle (**[D1]** §2)
Indexación NumPy directa	`X_bc.value[:, indices_activos]` selecciona columnas activas sin copiar el array completo

1.4 Resultado esperado y verificación

El resultado de esta evaluación real es comparable con el baseline (todas las características):

- **Baseline** ($|S| = D = 30$): $F = 0.9 \times \text{acc}_{CV5} - 0.1 \times 1.0$
- **Subconjunto óptimo**: mayor F que el baseline al seleccionar características informativas y reducir la penalización de parsimonia

La correctitud de la implementación se valida comparando la evaluación secuencial y la paralela con tolerancia $< 10^{-9}$ (véase Hito 4).

---

2. Hito 2: Integración del Conjunto de Datos

2.1 Selección del dataset

Se selecciona **WDBC (Wisconsin Diagnostic Breast Cancer)** como primer conjunto de datos del estudio experimental, coherente con la tabla de parámetros del documento de diseño del GA (**[D1]** §6):

Propiedad	Valor
Muestras (N)	569
Características (D)	30
Clases	2 (maligno / benigno)
Espacio de búsqueda	$2^{30} \approx 10^9$ subconjuntos
Fuente	`sklearn.datasets.load_breast_cancer()`

Justificación de la elección:

1. **Tamaño moderado:** 569 muestras permiten entrenar y evaluar clasificadores con rapidez, ideal para validar la arquitectura antes de escalar a datasets mayores (Arrhythmia con $D = 279$ o Gisette con $D = 5000$).
2. **Dimensionalidad adecuada:** $D = 30$ genera un espacio de búsqueda de $\sim 10^9$ subconjuntos, suficiente para que la búsqueda exhaustiva sea inviable y el GA demuestre su utilidad, pero manejable para depuración.
3. **Referencia en la literatura:** WDBC es uno de los datasets más empleados en estudios de selección de características con GA-wrapper (Aalaei et al., 2016; Huang et al., 2020), lo que permite contrastar resultados.
4. **Coincidencia con el diseño:** la longitud del cromosoma $D = 30$ coincide exactamente con la dimensionalidad del prototipo anterior, facilitando la transición.

2.2 Carga e inspección

El dataset se carga directamente desde scikit-learn sin necesidad de archivos externos:

```
from sklearn.datasets import load_breast_cancer

dataset = load_breast_cancer()
X = dataset.data    # (569, 30) - matriz de características
y = dataset.target  # 0=maligno, 1=benigno
D = X.shape[1]      # 30
```

Las 30 características corresponden a 10 medidas geométricas de los núcleos celulares (radio, textura, perímetro, área, suavidad, compacidad, concavidad, puntos cóncavos, simetría y dimensión fractal), cada una con tres estadísticos (media, error estándar y peor valor), produciendo $10 \times 3 = 30$ *features*.

---

3. Hito 3: Transmisión del Conjunto de Datos en Spark

3.1 Problema: copias redundantes a los workers

En el patrón `rdd.map(evaluar_fitness)`, Spark serializa la *closure* de la función y la envía a cada tarea. Si la función referencia directamente `X` e `y` como variables del Driver, Spark las copia con cada tarea. Para una población de $P = 100$ individuos en 8 particiones, esto supone 8 copias de `X` (569 × 30 × 8 bytes ≈ 1.1 MB por copia), desperdiciando memoria y ancho de banda.

3.2 Solución: broadcast variables

Siguiendo la arquitectura definida en el documento de Spark RDD (**[D2]** §2.5), se emplean *broadcast variables* para enviar los datos una sola vez a cada worker:

```
X_bc = sc.broadcast(X)
y_bc = sc.broadcast(y)
```

Dentro de la función de fitness, los datos se acceden mediante `X_bc.value` y `y_bc.value`. La variable broadcast se almacena una única vez en la memoria de cada executor (no por tarea), eliminando la redundancia.

3.3 Decisiones de implementación

Decisión	Alternativa	Justificación
Broadcast de arrays NumPy	Broadcast de DataFrames Spark	Los workers usan scikit-learn, que opera sobre NumPy; evitar conversiones.
Broadcast al inicio (una vez)	Broadcast por generación	Los datos no cambian; un solo broadcast basta para todo el ciclo evolutivo.
`destroy()` explícito al final	Dejar al GC	Libera memoria de los executors de forma determinista entre experimentos.

3.4 Patrón resultante

```
# Una vez al inicio
X_bc = sc.broadcast(X)
y_bc = sc.broadcast(y)

# En cada generación (la función accede al broadcast, no a X/y directamente)
rdd = sc.parallelize(poblacion, numSlices=NUM_CORES)
fitness_scores = rdd.map(evaluar_fitness).collect()

# Al finalizar
X_bc.destroy()
y_bc.destroy()
```

Este patrón es coherente con la recomendación del documento de arquitectura (**[D2]** §2.5): *"El clasificador tiene acceso a los datos de entrenamiento, que se difunden a los Workers mediante broadcast variables para evitar transferencias repetidas en cada generación."*

---

4. Hito 4: Evaluación Paralela del Fitness Real

4.1 Sustitución de la función fitness dummy

El prototipo del mes 2 usaba una función `fitness_costoso()` con `time.sleep()` para simular el coste del clasificador (véase **[D3]** §3.2). Se sustituye por una función que entrena un clasificador real:

Aspecto	Prototipo (mes 2)	Implementación real (mes 3)
Accuracy	Pseudo-accuracy determinista (hash)	GNB con 5-fold cross-validation
Coste	`time.sleep(50ms)`	Entrenamiento real de scikit-learn
Datos	No usa datos reales	WDBC via broadcast
Determinismo	Garantizado por hash	Garantizado por `StratifiedKFold`

4.2 Función de fitness integrada con Spark

La función sigue exactamente la fórmula definida en el documento de diseño del GA (**[D1]** §2):

$$\text{Fitness}(\mathbf{c}) = \alpha \cdot \text{Accuracy}_{CV5}(\mathbf{c}) - \beta \cdot \frac{|S|}{D}$$

Con $\alpha = 0.9$, $\beta = 0.1$ y $k = 5$ pliegues, conforme a la tabla de parámetros del diseño (**[D1]** §6, columna WDBC).

```
def evaluar_fitness(cromosoma):
    indices_activos = [i for i, gen in enumerate(cromosoma) if gen == 1]
    d = len(cromosoma)

    if len(indices_activos) == 0:
```

```
        return 0.0

    X_data = X_bc.value[:, indices_activos]
    y_data = y_bc.value

    gnb = GaussianNB()
    scores = cross_val_score(gnb, X_data, y_data, cv=CV_FOLDS, scoring='accuracy')
    accuracy = scores.mean()

    penalizacion = len(indices_activos) / d
    return round(ALPHA * accuracy - BETA * penalizacion, 6)
```

Decisiones clave:

1. **Clasificador GNB sin hiperparámetros:** coherente con el diseño (**[D1]** §5.1), que selecciona GNB por su velocidad $\mathcal{O}(N \times D)$ y ausencia de ajuste interno. Con $P = 100$, $G = 10$ y $k = 5$, una ejecución genera 5.000 entrenamientos.
2. **Caso especial del cromosoma nulo ($|S| = 0$):** retorna fitness $= 0$, conforme a la especificación del diseño (**[D1]** §2): *"si $|S| = 0$ (cromosoma nulo), el fitness se fija a 0 para penalizar soluciones inviables."*
3. **Selección de columnas con indexación NumPy:** `X_bc.value[:, indices_activos]` selecciona únicamente las columnas activas, sin copiar el array completo.
4. `cross_val_score` **de scikit-learn:** usa internamente `StratifiedKFold`, lo que mantiene la proporción de clases en cada pliegue. Esto es especialmente relevante para WDBC, donde la distribución es ligeramente desbalanceada (357 benignos vs. 212 malignos).

4.3 Comparación secuencial vs. paralela

El notebook `02_gnb_fitness_evaluation.ipynb` ejecuta la misma evaluación de forma secuencial (Python puro) y paralela (Spark RDD) para comparar tiempos y validar correctitud:

```
# Secuencial
fitness_seq = [evaluar_fitness(ind) for ind in poblacion]

# Paralelo
rdd = sc.parallelize(poblacion, numSlices=NUM_CORES)
fitness_par = rdd.map(evaluar_fitness).collect()
```

La validación de correctitud comprueba que ambos modos producen resultados idénticos (tolerancia $< 10^{-9}$), lo que confirma que la serialización de Spark y el broadcast no alteran los cálculos.

4.4 Ciclo evolutivo con fitness real

Se ejecuta un ciclo de 10 generaciones con los operadores genéticos definidos en el diseño (**[D1]** §3):

Operador	Tipo	Valor (WDBC)	Ref.
Selección	Torneo binario	Tamaño 2	**[D1]** §3.1
Cruce	Un punto	$p_{\text{cross}} = 0.8$	**[D1]** §3.2, §6
Mutación	Flip de bit	$p_{\text{mut}} = 0.2$, $p_{\text{mut_bit}} = 1/30$	**[D1]** §3.3, §6
Elitismo	Top E	$E = 2$	**[D1]** §6

Nota sobre la discrepancia con el diseño: El documento de diseño (**[D1]** §6) especifica $P = 50$ y $G = 50$ para WDBC. En esta implementación se usa $P = 100$ y $G = 10$ para equilibrar la exploración de la población con un tiempo de ejecución razonable durante la fase de desarrollo. Los parámetros definitivos se ajustarán en la fase de experimentación (mes 4).

El ciclo registra por generación: mejor fitness, fitness medio, peor fitness, número medio de características seleccionadas y tiempo de evaluación. Estos datos se visualizan en tres gráficos:

1. Evolución del fitness (mejor, medio, peor) por generación.
2. Evolución del número medio de características seleccionadas.
3. Tiempo de evaluación por generación.

4.5 Análisis del mejor individuo

Al finalizar el ciclo evolutivo, se analiza el mejor cromosoma encontrado:

- Se listan las características seleccionadas por nombre.
- Se recalcula la accuracy con 5-fold CV y se reporta la desviación estándar.
- Se compara con el *baseline* (todas las 30 características) para cuantificar la mejora en fitness y la reducción de dimensionalidad.

---

5. Hito 5: Experimento Inicial de Speed-Up

5.1 Diseño del experimento

El notebook `03_speedup_experiment.ipynb` implementa el primer experimento real del TFM, diseñado para responder a dos preguntas:

1. **¿Cómo escala el *speed-up* con el tamaño de la población?** A mayor P, el trabajo por partición aumenta y el overhead relativo de Spark debería disminuir.
2. **¿Cuál es el número óptimo de particiones?** Más particiones que cores pueden mejorar el balanceo de carga, pero aumentan el overhead de scheduling.

5.2 Variables experimentales

Variable	Valores	Justificación
Tamaño de población (P)	50, 100, 200, 500	$P = 50$ es el valor de diseño para WDBC (**[D1]** §6); 100–500 exploran escalabilidad
Particiones Spark	1, 2, 4, N_{cores}, $2 \times N_{\text{cores}}$	Desde secuencial-simulado (1) hasta sobre-particionado ($2 \times N_{\text{cores}}$)

5.3 Protocolo de medición

Cada combinación se mide siguiendo este protocolo:

1. **Generación determinista de la población:** `np.random.RandomState(seed=42)` garantiza reproducibilidad.
2. **Repeticiones:** $N_{\text{rep}} = 3$ mediciones por combinación; se reporta la **mediana** (robusta ante outliers por GC de la JVM o scheduling del SO).
3. **Baseline secuencial:** evaluación con `[evaluar_fitness(ind) for ind in poblacion]` (Python puro, un solo hilo).
4. **Evaluación paralela:** `sc.parallelize(poblacion, numSlices=n_part).map(evaluar_fitness).collect()`.
5. **Warm-up implícito:** la primera evaluación (test de verificación en §3) calienta la JVM y el broadcast antes de los experimentos.

5.4 Métricas capturadas

Métrica	Fórmula	Significado
t_{seq}	Tiempo secuencial	Baseline sin paralelismo
t_{par}	Tiempo paralelo	Incluyendo overhead de Spark
Speed-up (S)	$t_{\text{seq}}/t_{\text{par}}$	Factor de aceleración
Eficiencia (E)	$S/N_{\text{cores}} \times 100\%$	Aprovechamiento de los cores
Overhead	$t_{\text{par}} - t_{\text{seq}}/N_{\text{part}}$	Coste de serialización, scheduling y recolección
Tiempo por individuo	t/P (ms)	Coste marginal por cromosoma

5.5 Estructura de los sub-experimentos

Experimento 1 — Variación de P (particiones fijas = N_{cores}):

Se evalúa cómo crece el *speed-up* al aumentar la carga de trabajo. Con GNB sobre WDBC, cada evaluación individual es muy rápida ($\sim$ms), por lo que el overhead de Spark es proporcionalmente más significativo que con clasificadores costosos. Se espera que el *speed-up* mejore con P creciente, ya que el trabajo útil por partición aumenta respecto al coste fijo de scheduling.

Experimento 2 — Variación de particiones (P fijo = 100):

Se fija $P = 100$ (cercano al diseño) y se varía el número de particiones. Con N_{cores} cores físicos:

- **1 partición:** equivalente a secuencial dentro de Spark (un solo task).
- N_{cores} **particiones:** una tarea por core (configuración estándar).
- $2 \times N_{\text{cores}}$ **particiones:** sobre-particionado, que puede mejorar el balanceo si los cromosomas tienen costes de evaluación desiguales (cromosomas con pocos genes activos son más rápidos que los densos).

Experimento 3 — Mapa combinado $P\times$ particiones:

Cruza ambas variables para generar una vista completa del *speed-up* como un *heatmap* bidimensional. Permite identificar la combinación óptima y visualizar zonas donde el overhead de Spark domina (valores bajos de P y altos de particiones) frente a zonas de paralelismo eficiente.

5.6 Visualizaciones generadas

Experimento	Archivo	Contenido
Exp. 1 — Speed-up vs. P	`exp1_speedup_poblacion.png`	3 paneles: tiempos seq/par, curva speed-up vs. ideal, eficiencia
Exp. 2 — Speed-up vs. particiones	`exp2_speedup_particiones.png`	3 paneles: tiempos, curva speed-up vs. lineal, overhead
Exp. 3 — Mapa $P\times$ particiones	`exp3_heatmap_speedup.png`	2 paneles: heatmap speed-up y heatmap tiempo paralelo

5.7 Exportación de datos

Los resultados numéricos se exportan a CSV para su uso posterior en la fase de experimentación y redacción:

- `results/exp1_speedup_poblacion.csv`
- `results/exp2_speedup_particiones.csv`
- `results/exp3_speedup_combinado.csv`

---

5. Hitos 6 y 7: Primera Versión Completa del GA y Documentación

5.1 Objetivo (Hito 6)

Integrar en un único notebook todos los componentes del GA —selección, cruce, mutación, elitismo y evaluación paralela real— ejecutándolo durante hasta 50 generaciones y observando convergencia, evolución del fitness y reducción de características. El notebook de referencia es `04_ga_full_version.ipynb` (**[N3]**).

5.2 Parámetros definitivos del GA

Los parámetros se alinean exactamente con la tabla de diseño (**[D1]** §6, columna WDBC), corrigiendo la discrepancia de hitos anteriores ($P = 100$, $G = 10$):

Parámetro	Valor	Justificación
Tamaño población (P)	50	Valor de diseño para WDBC
Máx. generaciones (G_{max})	50	Suficiente para convergencia
Prob. cruce (p_{cross})	0.8	Estándar en literatura GA
Prob. mutación individuo (p_{mut})	0.2	Mantiene diversidad
Prob. flip por gen (p_{mut_bit})	$1/D$	≈ 1 bit alterado por cromosoma
Tamaño torneo (k)	5	Presión selectiva moderada
Elitismo (E)	2	Preserva los mejores
Prob. inicialización (p_{init})	0.5	Población inicial equiprobable
Tolerancia parada (ε_{stop})	0.001	Δfitness para estancamiento
Ventana parada (N_{stop})	10	Generaciones sin mejora significativa

5.3 Criterio de parada dual

El ciclo evolutivo se detiene cuando se cumpla **alguna** de estas dos condiciones:

1. G_{max} **alcanzado:** se han ejecutado 50 generaciones.
2. **Convergencia:** el mejor fitness no mejora más de $\varepsilon_{stop} = 0.001$ en las últimas $N_{stop} = 10$ generaciones.

```
if gen >= N_STOP:
    ventana = hist_mejor[-N_STOP:]
    delta = max(ventana) - min(ventana)
    if delta < EPS_STOP:
        break  # parada por convergencia
```

Este criterio, definido en **[D1]** §6, evita ciclos inútiles cuando el GA ha estancado su búsqueda.

5.4 Operadores genéticos modulares

Los operadores se implementan con una interfaz de estilo DEAP (*toolbox*), aunque DEAP no está instalado en el entorno y se usan implementaciones manuales con `random`. La nota de diseño (**[D1]** §3) prevé la sustitución futura por `toolbox.register(...)`:

Función	Descripción
`seleccion_torneo(pop, fit, k)`	Torneo de tamaño k; devuelve el mejor de k candidatos aleatorios
`cruce_un_punto(p1, p2)`	Intercambia colas a partir de un punto aleatorio
`mutacion_flip(ind, p_mut_bit)`	Invierte cada gen con probabilidad $p_{\text{mut_bit}}$
`elitismo(pop, fit, e)`	Copia los e mejores individuos sin modificar
`siguiente_generacion(pop, fit)`	Elitismo + torneo + cruce + mutación hasta reconstituir P individuos

5.5 Medición de tiempos por fase

A diferencia del notebook 02, aquí se mide por separado el tiempo de cada fase en cada generación:

- t_{eval} **(Workers):** tiempo del `sc.parallelize → map → collect`, que incluye la evaluación GNB con 5-fold CV.
- t_{ops} **(Driver):** tiempo de los operadores genéticos (selección, cruce, mutación, elitismo).

Esta separación permite cuantificar el reparto de coste entre la fase paralela y la fase secuencial del GA.

5.6 Visualizaciones generadas

El notebook produce un gráfico de cuatro paneles (`results/ga_full_convergencia.png`):

1. **Convergencia:** mejor fitness acumulado (monótonamente creciente) por generación.
2. **Evolución del fitness:** mejor, medio y peor por generación.
3. **Número medio de características** seleccionadas por generación.
4. **Reparto de tiempos:** barras apiladas t_{eval} (Workers, azul) + t_{ops} (Driver, rojo).

---

6. Decisiones Transversales y Coherencia con el Diseño

6.1 Parámetros del GA

Todos los parámetros utilizados en la implementación provienen directamente de la tabla del diseño del GA (**[D1]** §6, columna WDBC):

Parámetro	Diseño	Implementado	Nota
α	0.9	0.9	Sin cambio
β	0.1	0.1	Sin cambio
k (folds CV)	5	5	Sin cambio
p_{cross}	0.8	0.8	Sin cambio
p_{mut}	0.2	0.2	Sin cambio
$p_{\text{mut_bit}}$	$1/30$	$1/30$	Sin cambio
Elitismo (E)	2	2	Sin cambio
p_{init}	0.5	0.5	Sin cambio
P	50	100 (nb. 02)	Ampliado para explorar escalabilidad
G	50	10 (nb. 02)	Reducido en validación; corregido en Hito 6

6.2 Arquitectura Spark

La implementación sigue fielmente la arquitectura definida en el documento de Spark RDD (**[D2]** §2):

- **Evaluación de fitness en los Workers** (**[D2]** §2.1): cada worker recibe una partición de cromosomas, entrena GNB y retorna el fitness.
- **Operadores genéticos en el Driver** (**[D2]** §2.1): selección, cruce, mutación y elitismo se ejecutan secuencialmente, sin paralelismo.
- **Broadcast de datos** (**[D2]** §2.5): `X` e `y` se difunden una vez, antes del bucle generacional.
- **Patrón** `parallelize → map → collect` (**[D2]** §2.3): cada generación reconstruye el RDD a partir de la nueva población.

6.3 Clasificador

Se usa **Gaussian Naive Bayes** (`GaussianNB` de scikit-learn), conforme a la selección del diseño (**[D1]** §5.1). Las razones documentadas allí siguen vigentes en la implementación:

- Velocidad $\mathcal{O}(N \times D)$, crítica con $P \times G \times k$ entrenamientos.
- Sin hiperparámetros, eliminando variabilidad experimental.
- Estabilidad con subconjuntos dispersos en las primeras generaciones.

El clasificador secundario (LinearSVC, **[D1]** §5.2) no se implementa en este mes; se reserva para la fase de experimentación una vez convergido el GA.

---

7. Artefactos Generados

Notebooks

Notebook	Descripción
`01_parallel_fitness_prototype.ipynb`	Hito 1: prototipo con fitness simulado (`time.sleep`), validación del patrón Spark RDD
`02_gnb_fitness_evaluation.ipynb`	Hitos 2–4: dataset WDBC, broadcast, fitness real GNB, ciclo evolutivo 10 generaciones
`03_speedup_experiment.ipynb`	Hito 5: experimento de speed-up ($P\times$ particiones), exportación CSV
`04_ga_full_version.ipynb`	Hito 6: GA completo ($P = 50$, $G_{max} = 50$), parada dual, convergencia

Datos y gráficos (en `results/`)

Archivo	Contenido
`gnb_poblacion_inicial.png`	Distribución de fitness, fitness vs. nº características, genes activos (Hito 4)
`gnb_ga_evolucion.png`	Evolución fitness/características/tiempo, 10 generaciones (Hito 4)
`exp1_speedup_poblacion.png/.csv`	Speed-up por tamaño de población (Hito 5)
`exp2_speedup_particiones.png/.csv`	Speed-up por número de particiones (Hito 5)
`exp3_heatmap_speedup.png/.csv`	Mapa combinado $P\times$ particiones (Hito 5)
`ga_full_convergencia.png`	Convergencia, fitness, características y tiempos — 4 paneles (Hito 6)
`ga_full_series.csv`	Series temporales del GA por generación
`ga_full_experimento.json`	Resumen completo: hardware, Spark, parámetros, resultados
`ga_full_parametros.csv`	Tabla de parámetros del GA

---

8. Limitaciones Conocidas y Siguiente Paso

Limitaciones

1. **Operadores genéticos manuales:** los operadores (torneo, cruce un punto, mutación flip) se implementan con `random`, sin usar DEAP. El diseño (**[D1]** §3) prevé la integración con DEAP en una fase posterior; los operadores manuales tienen las mismas signaturas para facilitar esa migración.
2. **GNB sobre WDBC es rápido:** el coste por evaluación es del orden de milisegundos, lo que hace que el overhead de Spark sea proporcionalmente alto. Esto es una propiedad del dataset y del clasificador, no un defecto de la arquitectura. Con datasets más grandes (Arrhythmia $D = 279$, Gisette $D = 5000$) o clasificadores más costosos (LinearSVC), el speed-up mejorará significativamente.
3. **Ejecución en modo `local[*]`:** todos los experimentos se ejecutan en una sola máquina. El overhead de red de un cluster real no se captura, pero tampoco se dispone de más recursos de cómputo en esta fase.
4. **Solo WDBC:** en estos hitos se trabaja con un único dataset. La comparación entre datasets queda para la fase de experimentación (mes 4).

Siguiente paso

Con el GA completo validado y el experimento documentado, el siguiente paso es:

1. **Escalar al segundo dataset** (Arrhythmia, $D = 279$) y verificar que el speed-up mejora con mayor dimensionalidad.

2. **Incorporar LinearSVC** como clasificador de validación para el subconjunto S^* seleccionado (**[D1]** §5.2).
3. **Integrar DEAP** como librería de operadores genéticos, sustituyendo las implementaciones manuales.
4. **Comparar resultados entre datasets** para el capítulo experimental del TFM.

Referencias

Documentos internos del proyecto

Ref.	Fichero	Secciones relevantes
[D1]	`diseño_GA.md`	Fitness §2, operadores §3, GNB §5.1, LinearSVC §5.2, parámetros WDBC §6
[D2]	`arquitectura_spark_rdd_ga.md`	Driver/Workers §2.1, `parallelize`→`map`→`collect` §2.3, broadcast §2.5
[D3]	`prototipo_evaluacion_paralela.md`	Función fitness simulada con `time.sleep` §3.2

Notebooks

Ref.	Fichero	Contenido
[N1]	`01_parallel_fitness_prototype.ipynb`	Prototipo con fitness simulado, validación patrón Spark RDD
[N2]	`02_gnb_fitness_evaluation.ipynb`	Hitos 1–4: GNB, WDBC, broadcast, evaluación paralela, ciclo 10 gens
[N3]	`03_speedup_experiment.ipynb`	Hito 5: speed-up ($P\times$ particiones), exportación CSV
[N4]	`04_ga_full_version.ipynb`	Hito 6: GA completo ($P = 50$, $G_{max} = 50$), parada dual, convergencia